FROM MODEL DIFFS TO CAUSAL MECHANISMS

Using a CrossCoder to explain behavioral differences between base and fine-tuned code models

The case

The model solved the task—then kept writing.

A single latent feature helped control that continuation.

RESEARCH QUESTION

The evaluation tells us what changed. The CrossCoder asks why.

BEHAVIOR

- Base passes / fine-tuned fails
- Base fails / fine-tuned passes
- What kind of error occurred?

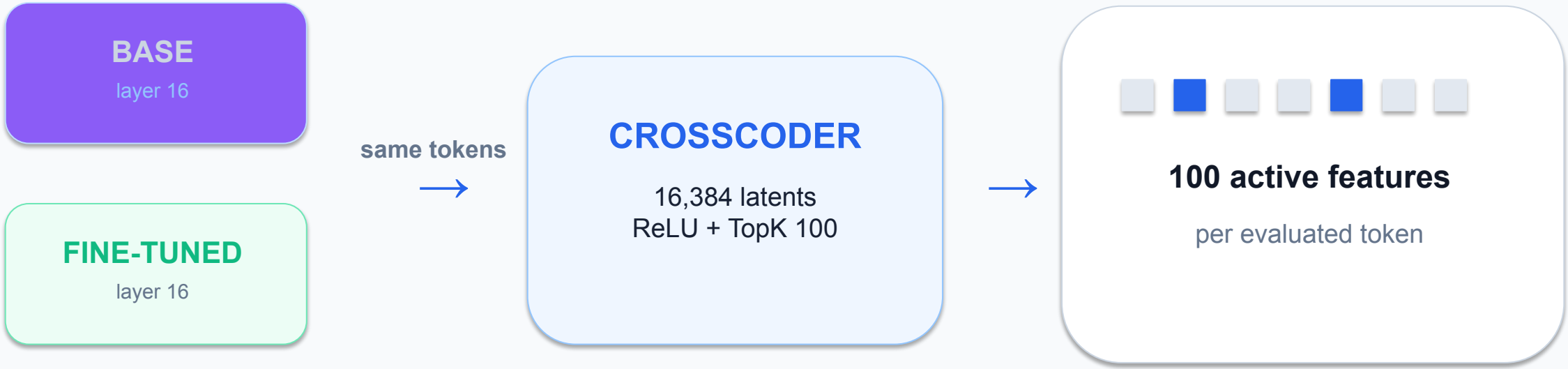
REPRESENTATION

- Paired layer-16 residuals
- Shared sparse features
- Decoder directions for intervention

Behavior localizes the difference. Features turn it into a testable hypothesis.

METHOD

First case study: DeepSeek base ↔ fine-tuned model diffing (DSTK100)



2,608 code texts · 617,959 evaluated tokens · same-text alignment

Method Lineage

Based on *Sparse Crosscoders for Cross-Layer Features and Model Diffing* (Lindsey et al., 2024). TopK sparsity follows the sparse-autoencoder design studied by Gao et al. (2024). **DSTK100** is our same-text, layer-16, TopK-100 implementation.

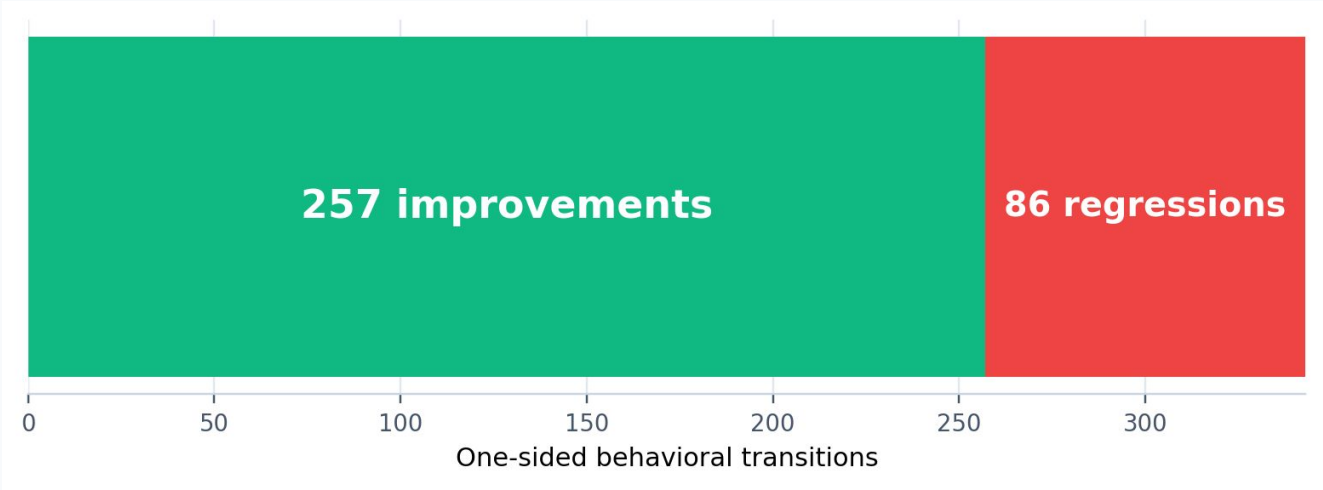
References

• Lindsey, J. et al. "Sparse Crosscoders for Cross-Layer Features and Model Diffing." *Transformer Circuits*, 2024.

• Gao, L. et al. "Scaling and Evaluating Sparse Autoencoders." 2024.

BEHAVIORAL MAP

Ours Results to DSTK100 agrees with ReCatcher



343

One-Sided Transitions

Total instances where models diverged

257

Improvements

base fail → FT pass

HumanEval+: 42
BigCodeBench: 215

86

Regressions

base pass → FT fail

HumanEval+: 7
BigCodeBench: 79

Agreement with ReCatcher

ReCatcher reported that Kotlin fine-tuning improved aggregate General Logic for DeepSeek-Coder, despite introducing localized regressions—particularly syntax errors.

ReCatcher · DeepSeek FT vs original

- HumanEval+: **+14.94%** in Incorrect Code / General Logic
- BigCodeBench: **+9.32%** in Incorrect Code / General Logic

Our paired evaluation

- HumanEval+: **+22.57 pp** in pass rate
- BigCodeBench: **+7.19 pp** in pass rate
- Improvements outnumbered regressions by **~3:1** overall

* The magnitudes are not directly comparable: ReCatcher used 10 generations per prompt and its own regression-percentage definition; our study used one canonical generation per task and official paired evaluation.

Abbassi et al., ReCatcher: Towards LLMs Regression Testing for Code Generation, 2025.

We ranked stable contrasts—not raw activation alone

4 summaries
max · early_max
mean · active fraction

→

Δ per task
aggregate(FT text)
− aggregate(base text)

→

Observed effect
mean Δ positives
− mean Δ controls

→

200 permutations
shuffle labels;
recompute effect

→

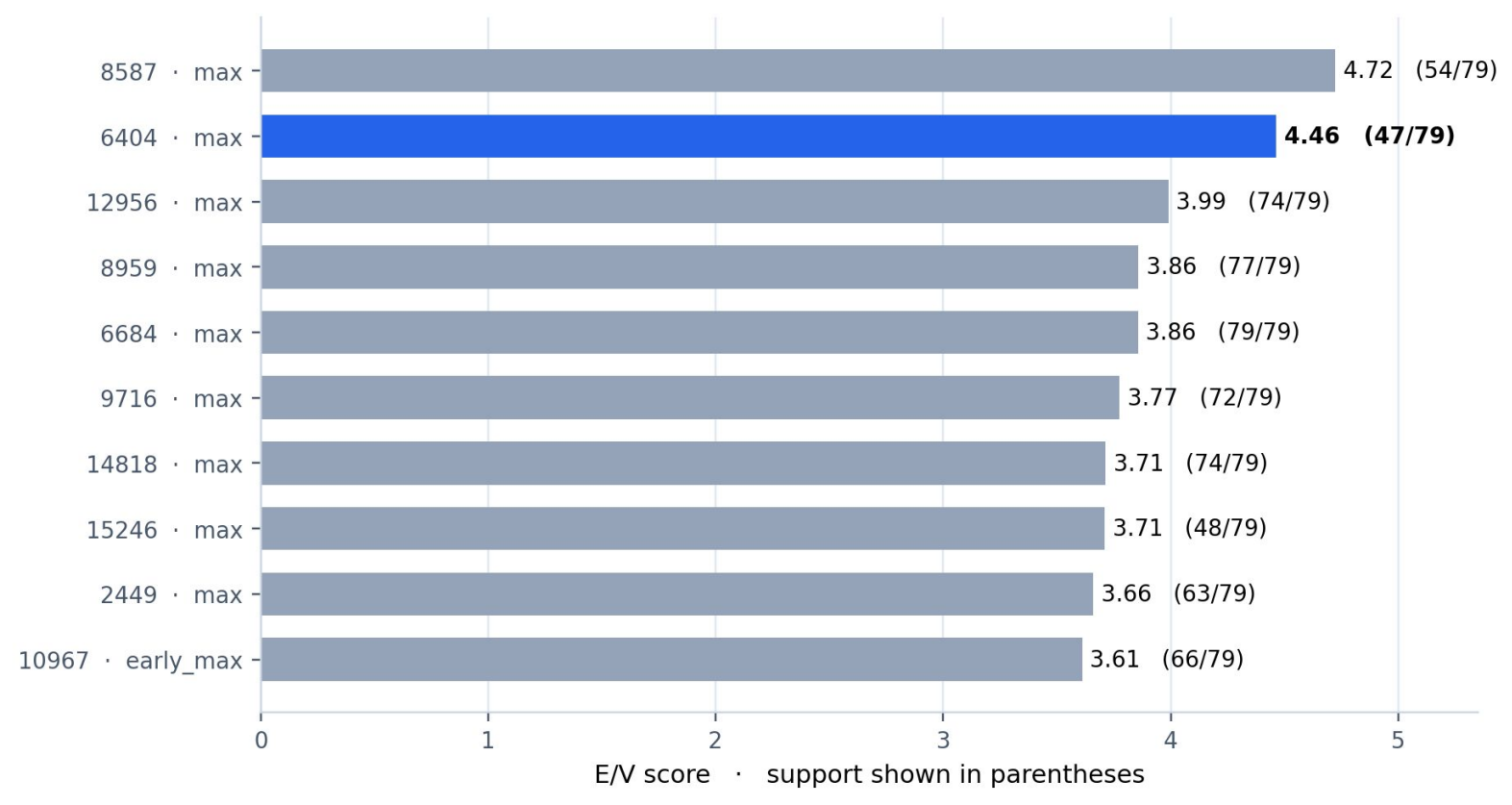
E/V
effect ÷ null SD

Minimum support **max(3 tasks, 10% of positives)**

E/V is a permutation signal-to-noise score—not a z-score or a p-value.

SCREENING RESULT

Feature 6404 ranked #2—and the result was specification-robust



6404

E/V 4.47

47 / 79 support

Same Top 10 with all four aggregations or E/V alone

A good causal candidate aligns meaning, failure mode, and timing

MEANING

What tokens and contexts activate the feature?

FAILURE MODE

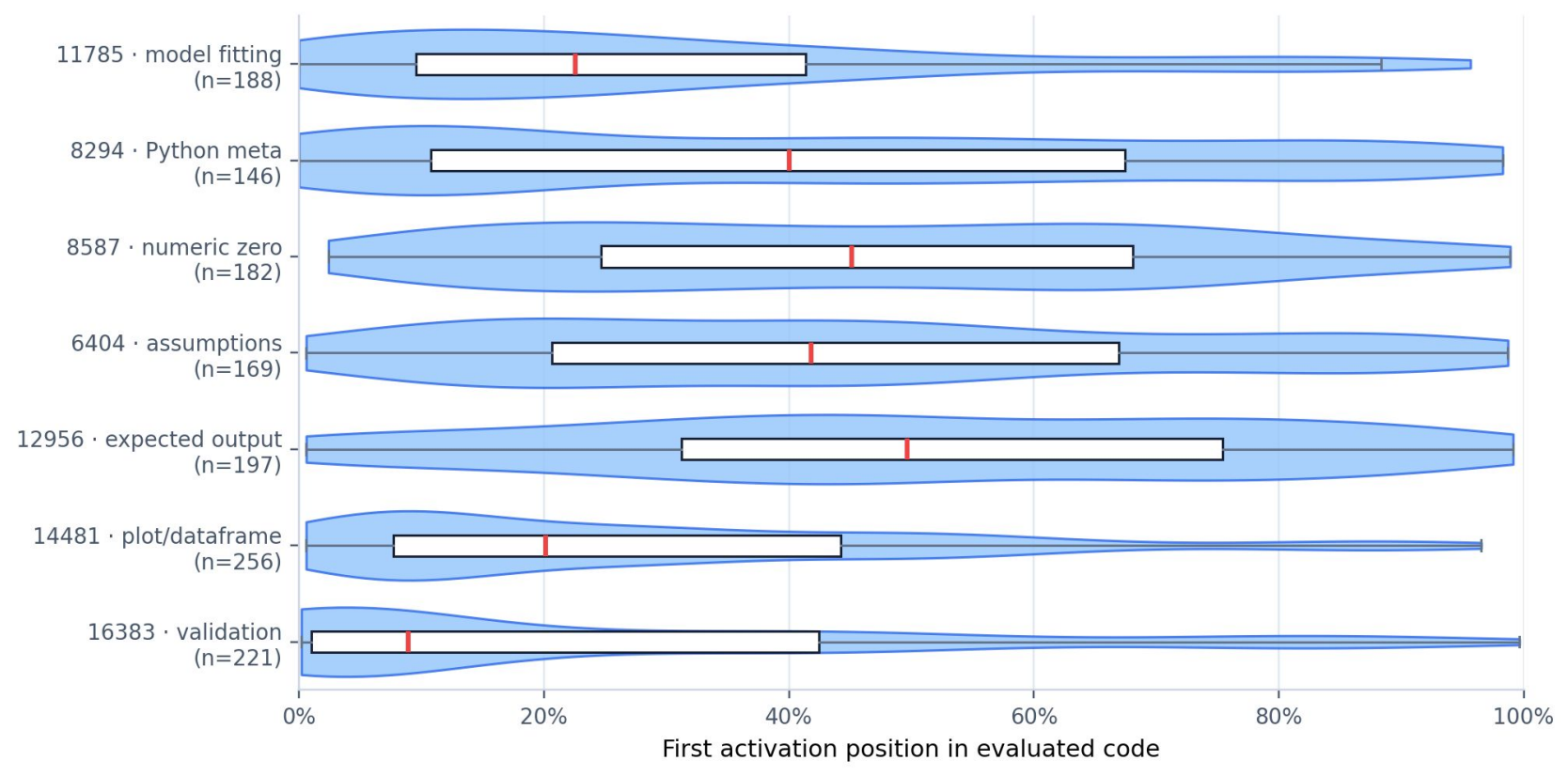
Which behavioral transition is it associated with?

TIMING

Does activation precede the relevant decision?

Semantic clarity alone is not enough: a late feature may be a consequence or style marker.

First-activation position separates plausible causes from late



Example

Feature 12956

Expected output / examples

Semantically clear—often too late to explain the implementation decision.

The pearl: many ‘improvements’ were simply cleaner stopping behavior

119 / 215

BigCodeBench improvements

55%

test/import contamination in the base output

```
 def task_func(...):
     ... plausible solution ...
-
-# tests
-from task_func import task_func
```

The fine-tuned model often won by stopping—not by implementing a different algorithm.

FEATURE INTERPRETATION

Feature 6404 matched the semantics of post-solution continuation

6404

Dominant activation contexts

- **expected**
- **should**
- **assumptions / caveats**
- **comments and boilerplate**

67% contamination cases

vs

31% other improvements

Exploratory odds ratio ≈ 4.44

Association generated the hypothesis; it did not establish causality.

A deliberately selected cohort turned the hypothesis into a test

1

Base failed

2

Fine-tuned passed

3

Base generated tests/imports

4

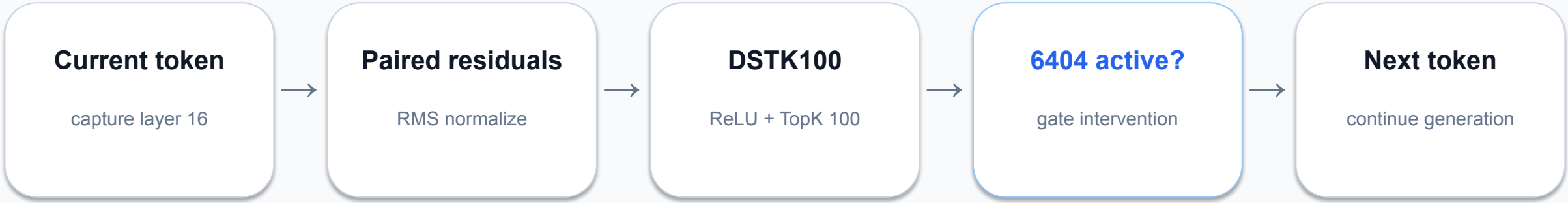
Feature 6404 was naturally active

80 **mechanistic cases**

Selected for mechanism—not an out-of-sample estimate.

ONLINE GATED STEERING

We intervened only when 6404 entered the natural TopK



$$h'_t = h_t + \alpha \cdot z_{6404,t} \cdot \mathrm{RMS}(h_t) \cdot d_{6404,\mathrm{base}}$$

The intervention changes the next-token distribution; it does not rewrite earlier tokens.

One example: the baseline solved the task, then imported its own test

BASELINE · FAIL

```
return []

# tests/test_task_func.py
import unittest
from task_func import task_func
```

The plausible function body is followed by test/import contamination.

6404 SUPPRESSION · PASS

```
return min_subsequence

# task_func({...})
# task_func({...})
```

Official pass; no generated test module import.

Red marks: online steps where feature 6404 entered the active TopK in the steered trajectory.

MAIN CAUSAL RESULT

The intervention recovered official passes—not just prettier text

19 / 80

fail → official pass

41 / 80

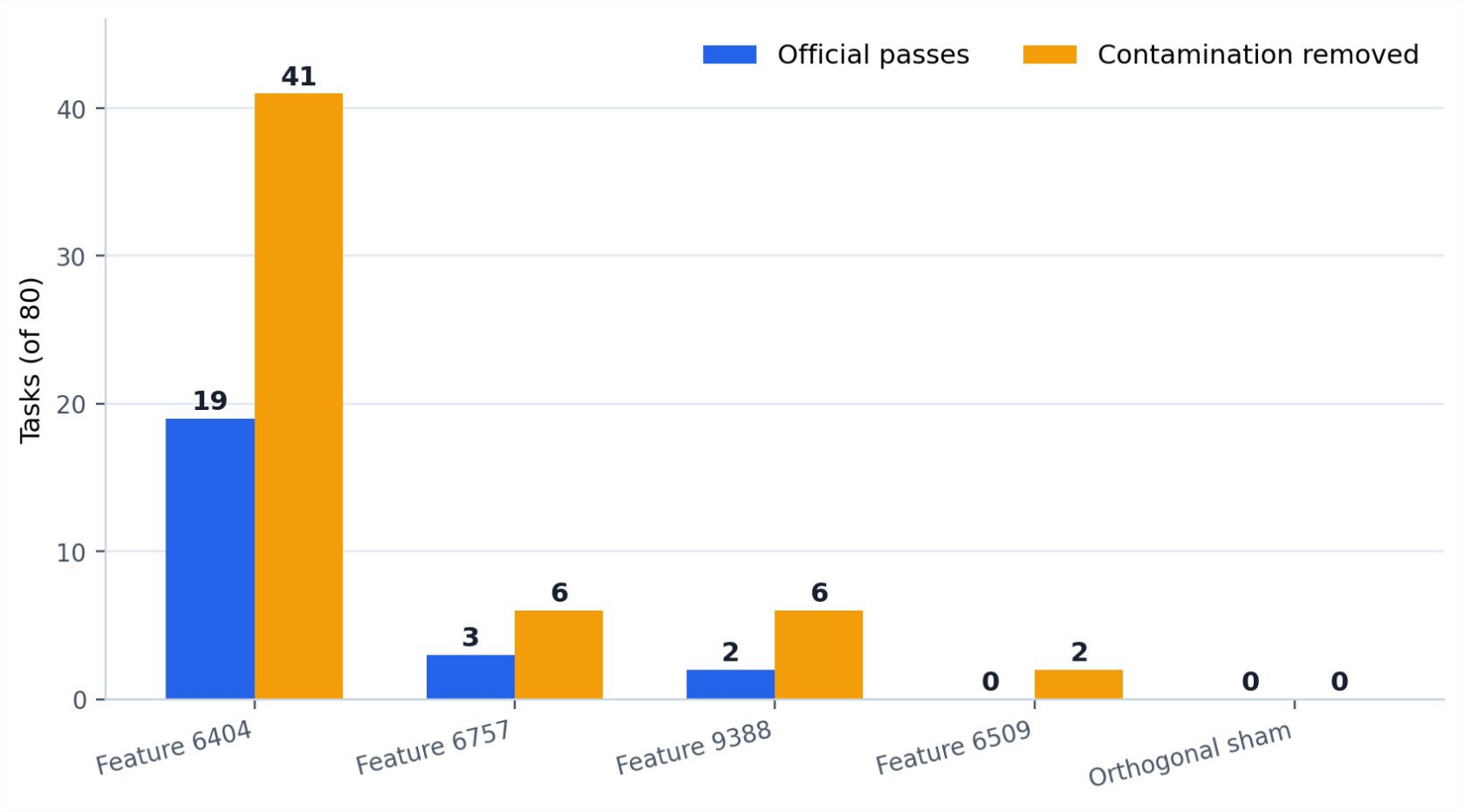
test/import contamination removed

α = −2 · TopK-gated · exact code evaluated with extraction v4

Cohort selected for this mechanism; do not read 23.75% as general benchmark gain.

SPECIFICITY

Matched controls isolate the 6404 direction



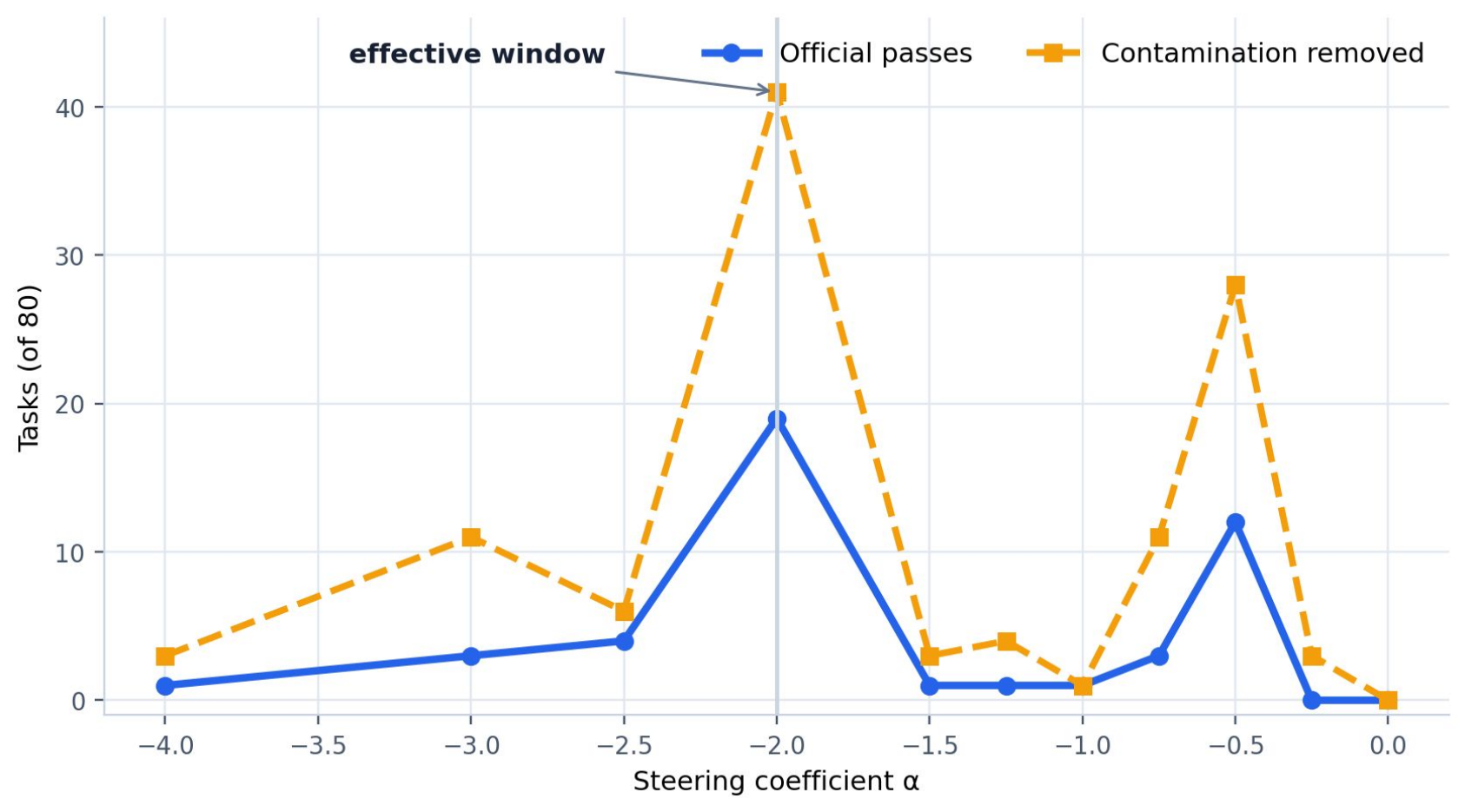
Controls matched

- decoder norm
- activation support
- temporal profile
- online gate activity

vs sham

pass $p \approx 3.8\times10^{-6}$
cleanup $p \approx 9.1\times10^{-13}$

Steering is not a volume knob



Why jagged?

- Token choice is discrete.
- One token changes the future context.
- Future gates then change.
- Pass/fail is discontinuous.
- One seed per task adds noise.